

Самостоятельная работа №2: Генерация и использование RSA ключей

1. Цель самостоятельной работы

Цель данной самостоятельной работы состоит в изучении и практической реализации алгоритма асимметричного шифрования RSA (Rivest–Shamir–Adleman) с использованием языка программирования Python. В ходе выполнения работы студенты научатся генерировать пары RSA ключей, шифровать и дешифровать сообщения, а также подписывать и проверять цифровые подписи.

2. Подробное техническое задание к самостоятельной работе

1. Установить библиотеку `pycryptodome` для работы с криптографическими алгоритмами в Python.
2. Создать скрипт на языке Python, который будет реализовывать следующие функции:
 - Генерация пары RSA ключей (публичного и приватного).
 - Шифрование заданного сообщения с использованием публичного ключа.
 - Дешифрование зашифрованного сообщения с использованием приватного ключа.
 - Создание цифровой подписи сообщения с использованием приватного ключа.
 - Проверка цифровой подписи с использованием публичного ключа.
3. Реализовать обработку ошибок и исключительных ситуаций.
4. Протестировать реализацию на нескольких примерах, убедившись в правильности шифрования, дешифрования, подписания и проверки подписей.

3. Список рекомендаций по использованию библиотек, структур данных, паттернов проектирования и так далее

Библиотеки

- `pycryptodome`: Библиотека для работы с криптографическими примитивами в Python. Установите её с помощью команды:

```
pip install pycryptodome
```

Структуры данных

- Используйте байтовые строки (bytes) для представления данных, которые будут шифроваться, дешифроваться и подписываться.
- Используйте словари для хранения ключей и других параметров шифрования, если это необходимо.

Паттерны проектирования

- **Функциональный подход:** Разделите ваш код на небольшие функции, каждая из которых выполняет конкретную задачу (например, генерация ключей, шифрование, дешифрование, подпись, проверка подписи).
- **Обработка исключений:** Используйте блоки `try-except` для обработки возможных ошибок при работе с криптографическими операциями.

4. Формат отчета по самостоятельной работе

Отчет по самостоятельной работе должен включать следующие разделы:

Введение

- Краткое описание цели и задач самостоятельной работы.

Теоретическая часть

- Описание алгоритма RSA и его основных принципов работы.
- Объяснение процесса генерации ключей, шифрования, дешифрования, подписания и проверки подписей.

Практическая часть

- Описание используемых инструментов и библиотек.
- Исходный код программы с комментариями.
- Скриншоты или вывод результатов тестирования программы.

Заключение

- Выводы по результатам выполнения самостоятельной работы.
- Описание возможных улучшений и доработок.

Приложения

- Дополнительные материалы, если таковые имеются (например, тестовые данные, дополнительный код).

Пример реализации в Python

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256

# Генерация пары RSA ключей
key = RSA.generate(2048)
private_key = key.export_key()
public_key = key.publickey().export_key()

def encrypt(message, public_key):
```

```

rsa_public_key = RSA.import_key(public_key)
cipher = PKCS1_OAEP.new(rsa_public_key)
encrypted_message = cipher.encrypt(message)
return encrypted_message

def decrypt(encrypted_message, private_key):
    rsa_private_key = RSA.import_key(private_key)
    cipher = PKCS1_OAEP.new(rsa_private_key)
    decrypted_message = cipher.decrypt(encrypted_message)
    return decrypted_message

def sign(message, private_key):
    rsa_private_key = RSA.import_key(private_key)
    h = SHA256.new(message)
    signature = pkcs1_15.new(rsa_private_key).sign(h)
    return signature

def verify(message, signature, public_key):
    rsa_public_key = RSA.import_key(public_key)
    h = SHA256.new(message)
    try:
        pkcs1_15.new(rsa_public_key).verify(h, signature)
        return True
    except (ValueError, TypeError):
        return False

# Тестирование реализации
plain_text = b'Hello, this is a test message!'
encrypted_text = encrypt(plain_text, public_key)
decrypted_text = decrypt(encrypted_text, private_key)
signature = sign(plain_text, private_key)
verification = verify(plain_text, signature, public_key)

print(f'Original text: {plain_text}')
print(f'Encrypted text: {encrypted_text}')
print(f'Decrypted text: {decrypted_text}')
print(f'Signature: {signature}')
print(f'Verification: {verification}')

```

Соблюдайте структуру отчета и приводите результаты тестирования, чтобы продемонстрировать правильность работы вашей реализации.